Software Impacts 27 (2026) 100814

Contents lists available at ScienceDirect

Software Impacts

journal homepage: www.journals.elsevier.com/software-impacts

Original software publication

E-STRANGE: A programming support platform in academia for code ethics, quality, and efficiency

Oscar Karnalim *, Yehezkiel David Setiawan

Faculty of Smart Technology and Engineering, Maranatha Christian University, Suria Sumantri Street No. 65, Bandung, Indonesia

ARTICLE INFO

Keywords:
Programming
Performance support
Code ethics
Code quality
Code efficiency
Assessments

ABSTRACT

In learning programming, students are often focused on the correctness of the programs. However, other important aspects should be considered, including code ethics, quality, and efficiency. This platform supports students in learning about these aspects through their own submissions. For each submission, a comprehensive report will be provided, showing instructors' expectations, simulated similarities, obvious similarities, the likelihood of AI-generated code, code quality issues, and the likelihood of inefficiency. Gamification is applied to promote engagement. More game points will be obtained by responding to relevant quizzes and submitting original, high-quality, and efficient programs.

Code metadata

Current code version	*V6*
Permanent link to code/repository used for this code version	https://github.com/SoftwareImpacts/SIMPAC-2025-347
Permanent link to reproducible capsule	
Legal code license	*Apache-2.0 license*
Code versioning system used	*git*
Software code languages, tools and services used	*PHP, Java, Python, MySQL*
Compilation requirements, operating environments and dependencies	
If available, link to developer documentation/manual	
Support email for questions	oscar.karnalim@it.maranatha.edu

Body of the article

1. Introduction

In academia, students should be supported in learning programming. The employment of software-related jobs is expected to increase by 15 percent from 2024 to 2034 [1]. Further, other disciplines now employ programming to complete their tasks [2], which promotes the integration of programming courses into non-computing disciplines [3, 4].

While program correctness is crucial, students should also consider code ethics [5], quality [6], and efficiency [7]. Several automated tools support students in learning these aspects. For code ethics, the tools include learning modules [8], plagiarism detectors [9], and similarity simulator [10]. For code quality, the tools include general-purpose code quality checkers [11], academic code quality checkers [12], and dedicated environments to learn code refactoring [13]. For code efficiency, the tools include runtime analyzers [14], complexity estimators [15], and dedicated environments to learn code cycles [16]. Nonetheless, they are dedicated to a particular aspect. Instructors might have difficulty promoting code ethics, quality, and efficiency simultaneously. They need to employ at least three tools, one for each aspect. Further, most of the tools cannot be easily integrated into classroom activities. They require either additional sessions or assessments.

To address the aforementioned gap, this platform provides an automated mechanism to simultaneously inform students about code ethics, quality, and efficiency. The platform works similarly to assessment completion in learning management systems, enabling smooth integration with any courses that include programming assessments.

* Corresponding author.
E-mail addresses: oscar.karnalim@it.maranatha.edu (O. Karnalim), if2172003@student.it.maranatha.edu (Y.D. Setiawan).

https://doi.org/10.1016/j.simpa.2026.100814
Received 29 December 2025; Accepted 28 January 2026

2665-9638/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

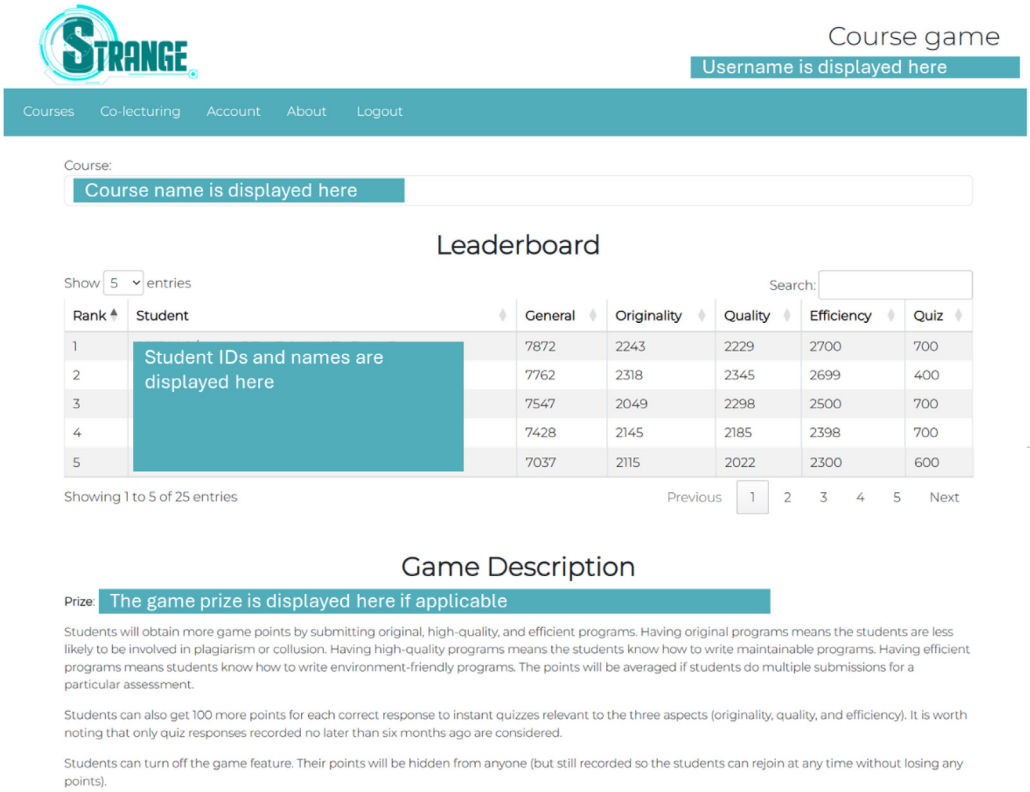

Fig. 1. Game dashboard.

2. Software overview

A comprehensive report will be provided for each submission, covering instructors' expectations, simulated similarities, obvious similarities, the likelihood of AI-generated code, code quality issues, and the likelihood of inefficiency. Instructors' expectations are displayed first, with focus on code ethics, quality, and efficiency. Students are expected to write their own programs that are original, high-quality and efficient.

Simulated similarities show that some changes do not really affect program meaning. The platform randomly takes several code blocks, highlights them in green, and disguises them. The disguises are limited to easily identifiable changes, such as in comments, white space, identifier names, and constants. Students are not expected to learn how to disguise their code from the simulated similarities.

Obvious similarities replace the simulated counterparts when a program is obviously similar to at least another already-submitted program based on Cosine similarity. Similar code segments are highlighted in red. It warns students that plagiarism and collusion can be identified and thus penalized.

The likelihood of AI-generated code is determined by the average difference between the submitted program and other already-submitted programs. A program is considered AI-generated if the average difference passes a particular threshold. AI-generated code tends to be unique and has its own syntax and style [17]. It should not be observed in student programs as AI-generated code is expected to be aligned to their own needs before being used.

Code quality issues relate to Java and Python syntax. The identified issues will be highlighted in green. Once clicked, the issue and its fixing strategy will be provided. This is expected to help students understand the criteria for high-quality code.

The likelihood of inefficiency is defined in terms of program size. A program should be comparable in size to other already-submitted programs. Longer programs tend to be less efficient. Predicting runtime or time complexity is not applicable since they are complex and time-consuming. Further, they are not perfectly accurate.

To further promote student engagement, gamification is applied through the promotion of original, high-quality, and short programs. Students will earn more game points by submitting programs that meet these criteria. Prizes can be provided for students with high game points if needed. The game leaderboard can be seen in Fig. 1. Students will be ranked based on their obtained game points. For each student, the total game points are displayed, broken down into four categories: originality, quality, efficiency, and quiz. At the end of the page, the game description and the prize information are provided.

For the uniqueness part, a program will receive 100 game points if its average similarity to other programs does not exceed a particular threshold. Otherwise, it will be 100 minus the average similarity degree. The game points of multiple submissions to one assessment will be averaged.

For the quality part, a program will receive game points equal to 100 minus the proportion of identified code quality issues to the total number of code blocks. Each code block is assumed to be eight program statements. It employs the same mechanism as that of the uniqueness part for handling multiple submissions.

For the efficiency part, a program will receive 100 game points if its size is comparable to that of other already-submitted programs. Otherwise, it will be assigned by 100 minus the difference from the average size of already-submitted programs. The game points of multiple submissions to one assessment will be averaged.

For the quiz part, students are expected to complete weekly quizzes to reflect on their knowledge of code ethics, quality, and efficiency. Each quiz consists of one multiple-choice question about the topics, randomly selected from the quiz bank. One correct response entails 100 game points.

3. Impact overview

The platform's prototypes have been evaluated in the past [18–20] and have been used in more than 30 classes at three institutions since 2020. To date, it serves more than 750 students. Two-thirds are computing students: around 500 undergraduates and ten graduates. The rest are non-computing students, including 200 undergraduates and 30 high school students. Nevertheless, the code of our platform and its prototypes have not been released to the public. Further, at that time, the user interface was less user-friendly and responsive. According to the experiments, students are more aware of code ethics and quality. They are also less likely to engage in academic misconduct or to cause code quality issues. Currently, the platform works with submissions written in Java and Python, two common programming languages [21].

While some quasi-experiments have been conducted to evaluate the platform's prototypes, they focus only on conventional code ethics or code quality. The platform's impact on AI code ethics, its impact on efficiency, and its overall impact across all three aspects remain unexplored. We are now addressing the first two. Further, all experiments were conducted in programming courses that employed weekly assessments, hosted at institutions in a particular city. Experiments can be reconducted across courses with different assessment designs, preferably hosted by institutions from different areas and educational backgrounds.

Our platform has some limitations which can be addressed in future work. First, while instructors' expectations for code ethics, quality, and efficiency on our platform are common, we are aware that they cannot be generalized across all courses and assessments. Future work might enable configurable settings to cater to various needs. Second, our detection of similarities and uniqueness is quite simple. Employing other similarity measurements for that might enrich the findings. Third, the identified code quality issues might not cover all instructors' needs. They need to be reconfigured in the future. Fourth, the detection of efficiency is somewhat naïve and can be replaced.

CRediT authorship contribution statement

Oscar Karnalim: Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Conceptualization, Methodology, Validation, Supervision, Writing – original draft, Writing – review & editing. **Yehezkiel David Setiawan:** Software, Validation, Writing – Original Draft.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Oscar Karnalim reports financial support and article publishing charges were provided by Maranatha Christian University. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

To the Institute for Research and Community Service, Maranatha Christian University, Indonesia, for supporting the research.

References

[1] Bureau of labor statistics USD of L software developers, quality assurance analysts, and testers, in: Occupational Outlook Handbook, U.S. Bureau of Labor Statistics, https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm#tab-6. (Accessed 25 November 2025).

[2] S.V. Kovalchuk, V.V. Krzhizhanovskaya, M. Paszyński, et al., Computational science for a better future, J. Comput. Sci. 62 (2022) 101745, http://dx.doi.org/10.1016/J.JOCS.2022.101745.

[3] K.A. Mills, J. Cope, L. Scholes, L. Rowe, Coding and computational thinking across the curriculum: A review of educational outcomes, Rev. Educ. Res. 95 (2025) 581–618, http://dx.doi.org/10.3102/00346543241241327.

[4] D.P. Bills, R.L. Canosa, Sharing introductory programming curriculum across disciplines, in: ACM Information Technology Education Conference, ACM, 2007, pp. 99–106.

[5] I. Albluwi, Plagiarism in programming assessments: a systematic review, ACM Trans. Comput. Educ. 20 (2019) 6:1–6:28.

[6] H. Keuning, B. Heeren, J. Jeuring, Code quality issues in student programs, in: ACM Conference on Innovation and Technology in Computer Science Education, 2017, pp. 110–115.

[7] Y. Tao, W. Chen, Q. Ye, Zhao. Y, Beyond functional correctness: An exploratory study on the time efficiency of programming assignments, in: International Conference on Software Engineering, IEEE Computer Society, 2024, pp. 320–330.

[8] H.H. Tsang, A.S. Hanbidge, Tin. T, Experiential learning through inter-university collaboration research project in academic integrity, in: 23rd Western Canadian Conference on Computing Education, 2018.

[9] R. Maertens, M. Van Neyghem, M. Geldhof, et al., Discovering and exploring cases of educational source code plagiarism with dolos, SoftwareX 26 (2024) 101755, http://dx.doi.org/10.1016/J.SOFTX.2024.101755.

[10] T. Le, A. Carbone, J. Sheard, et al., Educating computer programming students about plagiarism through use of a code similarity detection tool, in: 2013 International Conference on Learning and Teaching in Computing and Engineering, Macau, 2013, pp. 98–105.

[11] E.A. Alomar, Nurturing code quality: Leveraging static analysis and large language models for software quality in education, ACM Trans. Comput. Educ. 25 (16) (2025) http://dx.doi.org/10.1145/3722229.

[12] L.C. Ureel II, C. Wallace, Automated critique of early programming antipatterns, in: 50th ACM Technical Symposium on Computer Science Education, 2019, pp. 738–744.

[13] H. Keuning, B. Heeren, J. Jeuring, A tutoring system to learn code refactoring, in: 52nd ACM Technical Symposium on Computer Science Education, 2021, pp. 562–568.

[14] F. Chen, T.F. Şerbănuţă, G. Roşu, JPredictor: A predictive runtime analysis tool for java, Proc. - Int. Conf. Softw. Eng. 22 (2008) 1–230, http://dx.doi.org/10.1145/1368088.1368119.

[15] T. Dobravec, Estimating the time complexity of the algorithms by counting the java bytecode instructions, in: 2017 IEEE 14th International Scientific Conference on Informatics, 2017, pp. 74–79, http://dx.doi.org/10.1109/INFORMATICS.2017.8327225.

[16] Y. Aliev, G. Ivanova, A. Borodzhieva, Developing educational software models for teaching cyclic codes in coding theory, Appl. Sci. 15 (9604) (2025) http://dx.doi.org/10.3390/APP15179604.

[17] A. Pang, F. Vahid, ChatGPT and cheat detection in CS1 using a program autograding system, in: Annual Conference on Innovation and Technology in Computer Science Education, ITiCSE, Association for Computing Machinery, 2024, pp. 367–373.

[18] O. Karnalim, Chivers W. Simon, Gamification to help inform students about programming plagiarism and collusion, IEEE Trans. Learn. Technol. (2023) 1–14.

[19] O. Karnalim, Chivers W. Simon, B.S. Panca, Educating students about programming plagiarism and collusion via formative feedback, ACM Trans. Comput. Educ. 22 (2022) 31:1–31:31.

[20] O. Karnalim, Chivers W. Simon, B.S. Panca, Automated reporting of code quality issues in student submissions, in: IFIP WCCE 2022: World Conference on Computers in Education, Springer, Hiroshima, 2022.

[21] Mason R. Simon, T. Crick, et al., Language choice in introductory programming courses at Australasian and UK universities, in: 49th ACM Technical Symposium on Computer Science Education, 2018, pp. 852–857.